

AI Root Cause Analysis Platform

Project Overview • Business Problem • Solution • Architecture • Technology Stack

Purpose: A practical project definition for a Data Scientist / AI Engineer team. The document explains what business problem the platform solves, how the product works, the proposed repository structure, development environment, and selected technologies.

Product concept: Automatically detect unusual KPI behavior, investigate possible drivers across business dimensions, validate findings analytically, and explain the result to business users using an AI layer.

1. Business Problem

Companies collect data from ERP, CRM, finance, sales, inventory, and operational systems. The problem is not simply having data; the problem is quickly understanding **why a business metric changed**.

For example, a manager notices that revenue dropped in July and asks: **“Why did revenue decrease?”**

Traditional investigation

```
Manager notices KPI ↓
↓
Ask Data Analyst / Data Scientist
↓
Extract data with SQL
↓
Clean / filter data
↓
Create charts
↓
Compare periods
↓
Analyze Region / Product / Customer
↓
Investigate correlations
↓
Write explanation
```

Result: manual, slow, and dependent on analyst availability.

Business impact

The investigation can consume significant analyst time, especially when the business needs to check many dimensions. Different analysts may also use different investigation approaches, making the process difficult to standardize and reproduce.

2. Proposed Solution

The platform provides an automated analytical workflow. A user selects a KPI and comparison period, and the system investigates the change using deterministic Data Science tools. The AI layer then converts the validated analytical evidence into a business explanation.

Proposed solution

```
Dataset / SQL Server
↓
Schema Validation
↓
Data Profiling
↓
KPI Detection
↓
KPI Selection
↓
Anomaly Detection
↓
Dimension Analysis
↓
Contribution Analysis
↓
Statistical Validation
↓
Root Cause Ranking
↓
Evidence / RCA Tree
↓
AI Explanation with Ollama
```

Example

The system could identify that revenue decreased by 14.8%, then determine that the largest contributing segment was Product A in Cairo among Enterprise customers. The result should include the measured contribution and supporting evidence rather than only a natural-language guess.

3. End-to-End Product Flow

1. User uploads CSV / Excel or connects to SQL Server
2. System validates the dataset
3. System profiles the dataset
4. System detects possible KPIs, time fields, and dimensions
5. User selects KPI + aggregation + time field + comparison + dimensions
6. RCA engine investigates the KPI change
7. Statistical engine validates candidate drivers
8. System creates an RCA tree and evidence
9. Ollama explains the validated findings
10. Frontend displays the RCA investigation

4. Data Foundation

Schema Validation

Answers: **Is this dataset structurally usable?** The validator checks data types, convertible values, critical fields, invalid values, and analysis prerequisites. The result should be PASS, WARNING, or BLOCKED.

Data Profiling

Answers: **What is inside the dataset?** The profile includes row/column counts, missing values, duplicates, numeric statistics, categorical frequencies, datetime ranges, and other quality indicators.

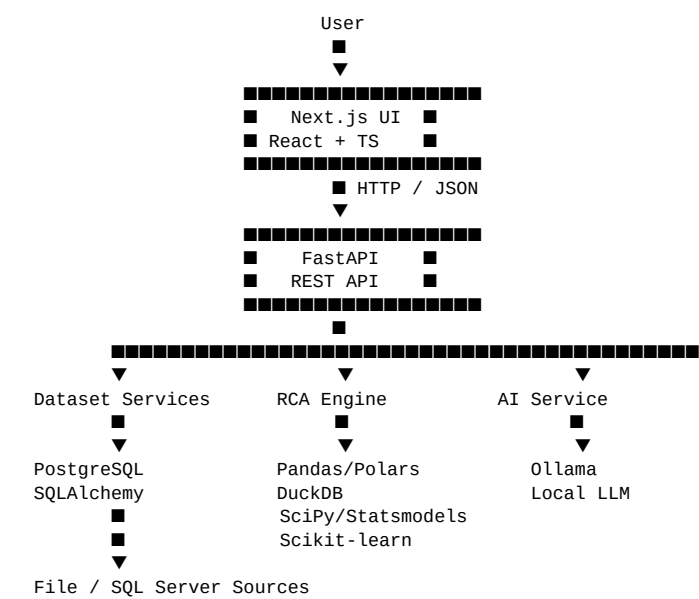
KPI Selection

Profiling identifies numeric measures, datetime candidates, and categorical dimensions. The user then defines the analytical contract: KPI, aggregation, time dimension, comparison period, and dimensions to investigate.

KPI Definition example

```
{
  "name": "Revenue",
  "column": "revenue",
  "aggregation": "SUM",
  "time_column": "date",
  "dimensions": [
    "region",
    "product",
    "customer_segment"
  ],
  "comparison": "previous_period"
}
```

5. High-Level System Architecture



Architecture principle

The LLM should not replace deterministic analysis. Python analytical tools calculate metrics, statistical tests, contributions, and rankings. Ollama receives validated results and explains them. This separation reduces hallucination risk and makes RCA findings auditable.

6. Backend Technology Stack

Technology	Purpose
Python	Primary backend and Data Science language.
FastAPI	REST API, request handling, validation integration, and service endpoints.
Pydantic	Typed request/response schemas and configuration validation.
SQLAlchemy	Database ORM/data-access layer for PostgreSQL and application persistence.
DuckDB	Fast analytical SQL engine for local/in-process dataset analysis.
Pandas / Polars	Data manipulation, profiling, transformations, and analytical processing.
SciPy	Statistical tests and numerical analysis.
Statsmodels	Statistical modeling, time-series analysis, and diagnostics.
Scikit-learn	Machine learning, anomaly detection, preprocessing, and model utilities.

7. Frontend Technology Stack

Technology	Purpose
Next.js	Frontend application framework, routing, rendering, and application structure.
React	Component-based UI.
TypeScript	Type safety and maintainable frontend contracts.
Tailwind CSS	Utility-first styling and responsive UI.
shadcn/ui	Reusable UI components and consistent interface patterns.
Plotly	Interactive KPI, trend, contribution, and RCA visualizations.

8. AI and Database

Ollama: provides the local LLM layer for interpreting validated RCA evidence and generating natural-language explanations. The exact Ollama model can be selected according to available hardware and quality requirements.

PostgreSQL: stores application data such as users, companies, dataset metadata, KPI definitions, investigation records, and other persistent application state. PostgreSQL can also support vector data later if an embedding/RAG capability is introduced.

9. Project Repository Structure

The repository separates frontend and backend responsibilities. Docker configuration and shared repository configuration live at the root.

```
ai-root-cause-analysis/
├── backend/
│   ├── app/
│   │   ├── api/
│   │   │   ├── routes/
│   │   │   │   ├── auth.py
│   │   │   │   ├── datasets.py
│   │   │   │   ├── uploads.py
│   │   │   │   ├── sql_connections.py
│   │   │   │   ├── sql_editor.py
│   │   │   │   ├── rca.py
│   │   │   └── router.py
│   │   ├── core/
│   │   │   ├── config.py
│   │   │   ├── security.py
│   │   │   ├── logging.py
│   │   │   └── exceptions.py
│   │   ├── db/
│   │   │   ├── models/
│   │   │   ├── repositories/
│   │   │   ├── session.py
│   │   │   └── migrations/
│   │   ├── schemas/
│   │   │   ├── auth.py
│   │   │   ├── dataset.py
│   │   │   ├── upload.py
│   │   │   ├── profiling.py
│   │   │   └── rca.py
│   │   ├── services/
│   │   │   ├── dataset_service.py
│   │   │   ├── profiling_service.py
│   │   │   ├── validation_service.py
│   │   │   ├── sql_service.py
│   │   │   └── storage_service.py
│   │   ├── storage/
│   │   │   ├── base.py
│   │   │   └── local.py
│   │   ├── connectors/
│   │   │   └── sqlserver.py
│   │   ├── analytics/
│   │   │   ├── profiling/
│   │   │   ├── schema/
│   │   │   │   ├── anomaly_detection.py
│   │   │   │   ├── dimension_analysis.py
│   │   │   │   ├── contribution.py
│   │   │   │   ├── statistical_validation.py
│   │   │   │   └── ranking.py
│   │   ├── ai/
│   │   │   ├── ollama_client.py
│   │   │   ├── prompts/
│   │   │   └── rca_explainer.py
│   │   ├── tests/
│   │   │   ├── unit/
│   │   │   └── integration/
│   │   ├── requirements.txt
│   │   ├── Dockerfile
│   │   └── .dockerignore
├── frontend/
│   ├── src/
│   │   ├── app/
│   │   │   ├── components/
│   │   │   ├── features/
│   │   │   ├── datasets/
│   │   │   ├── profiling/
│   │   │   └── kpi-selection/
```

```

■ ■ ■ ■ ■ sql-editor/
■ ■ ■ ■ ■ rca/
■ ■ ■ ■ ■ lib/
■ ■ ■ ■ ■ hooks/
■ ■ ■ ■ ■ types/
■ ■ ■ ■ ■ public/
■ ■ ■ ■ ■ package.json
■ ■ ■ ■ ■ Dockerfile
■ ■ ■ ■ ■ .dockerignore
■
■ ■ ■ ■ ■ docker/
■ ■ ■ ■ ■ nginx/
■ ■ ■ ■ ■ nginx.conf
■
■ ■ ■ ■ ■ docker-compose.yml
■ ■ ■ ■ ■ .env.example
■ ■ ■ ■ ■ .gitignore
■ ■ ■ ■ ■ README.md
■ ■ ■ ■ ■ LICENSE

```

10. Python Virtual Environment

Development should use a Python virtual environment so project dependencies are isolated from the system Python installation. The virtual environment should not be committed to Git.

```

backend/
■ ■ ■ ■ ■ .venv/          # local only, NOT committed
■ ■ ■ ■ ■ requirements.txt
■ ■ ■ ■ ■ app/

Create:
python -m venv .venv

Activate on Windows:
.venv\Scripts\activate

Activate on Linux/macOS:
source .venv/bin/activate

Install:
pip install -r requirements.txt

Run:
uvicorn app.main:app --reload

```

11. Backend requirements.txt

The dependency file should contain the runtime packages required by the backend. Version pinning can be added after the initial environment is validated.

```

fastapi
uvicorn[standard]
pydantic
pydantic-settings
sqlalchemy
psycopg[binary]
duckdb
pandas
polars
scipy
statsmodels
scikit-learn
python-multipart
httpx

# SQL Server connectivity
pyodbc

# Optional migrations
alembic

# Ollama HTTP client
ollama

```

12. Docker Structure

Docker should provide reproducible application environments. The backend and frontend should each have their own Dockerfile. docker-compose.yml should orchestrate the local development stack.

```

docker-compose.yml
  ■
  ■■■ backend
  ■   ■■■ backend/Dockerfile
  ■
  ■■■ frontend
  ■   ■■■ frontend/Dockerfile
  ■
  ■■■ postgres
  ■
  ■■■ optional nginx

```

Backend Dockerfile responsibilities

Install Python dependencies, copy backend source, expose the FastAPI service, and start the application with Uvicorn. The Docker image should not contain local secrets or development uploads.

Frontend Dockerfile responsibilities

Install Node dependencies, build the Next.js application, and run the application using the selected production/development strategy.

13. .gitignore

The repository must not commit secrets, virtual environments, local uploads, dependency folders, generated build files, logs, or local database artifacts.

```

# Python
__pycache__/
*.py[cod]
.venv/
venv/

# Secrets
.env
.env.*
!.env.example

# Local data
uploads/
storage/
docker-data/
*.db

# Testing / cache
.pytest_cache/
.mypy_cache/
.coverage
htmlcov/

# Frontend
node_modules/
.next/
dist/
build/

# Logs
*.log
logs/

# IDE / OS
.vscode/
.idea/
.DS_Store
Thumbs.db

```

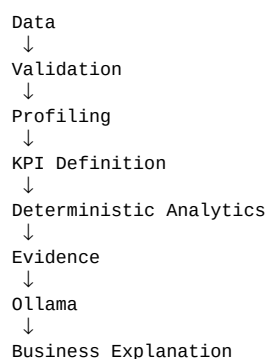
14. Separation of Responsibilities

Layer	Responsibility
Frontend	User interaction, configuration, data visualization, RCA presentation.
FastAPI	API boundary, authentication, request/response handling, orchestration.
Services	Application workflows such as uploads, profiling, validation and datasets.
Analytics	Deterministic Data Science calculations.

Layer	Responsibility
AI	Interpret validated evidence and produce natural-language explanations.
PostgreSQL	Persistent application metadata and state.
DuckDB	Local analytical SQL over datasets.
Storage	Physical dataset files behind a storage abstraction.

15. Core Engineering Principle

Do not build an AI chatbot around the data. Build an analytical product with an AI explanation layer.



This architecture keeps the Data Science logic testable and reproducible while allowing the AI layer to make the product easier for business users to understand and interact with.

16. MVP Definition

The MVP should demonstrate the complete path from data to explanation: upload or connect data, validate it, profile it, select a KPI and dimensions, run an initial RCA analysis, and present evidence-backed results through the Next.js interface. Ollama should explain the analytical findings without inventing calculations.